

FRIEND ФУНКЦИИ И КЛАССЫ

Семинар 12

friend функции

Дружественная функция – это функция, которая имеет доступ к закрытым членам класса как если бы она сама была членом этого класса.

Дружественно функцией может быть как обычная функция так и метод класса.

friend функции

Для объявления дружественной функции используется ключевое слово **friend** перед прототипом функции, которую вы хотите сделать дружественной классу. Неважно, в `public` или `private` вы ее объявите.

Функции - “друзья” не нарушают принцип инкапсуляции, потому что класс сам решает, кто ему друг.

friend функции

```

3 //=====
4 #include <stdexcept>
5
6 class Fraction
7 {
8 public:
9     Fraction(int num, int den);
10
11     friend double devision(const Fraction& fraction);
12
13 private:
14     int _numerator;
15     int _denominator;
16 };
17
18 Fraction::Fraction(int num, int den)
19 {
20     if (!den)
21         throw std::runtime_error("Denominator cannot be 0");
22     _numerator = num;
23     _denominator = den;
24 }
25 //=====
26 double devision(const Fraction& fraction)
27 {
28     return static_cast<double>(fraction._numerator) / fraction._denominator;
29 }
30 //=====

```

```

31 int main()
32 {
33     int n = 0, d = 0;
34     std::cin >> n >> d;
35     try
36     {
37         Fraction fract(n, d);
38         std::cout << devision(fract) << "\n";
39     }
40     catch (std::exception const& exc)
41     {
42         std::cout << exc.what() << '\n';
43     }
44 }

```

```

chernousov@DESKTOP-Igor:~/ex$ g++ exception.cpp
chernousov@DESKTOP-Igor:~/ex$ ./a.out
5 2
2,5
chernousov@DESKTOP-Igor:~/ex$ ./a.out
4 16
0.25
chernousov@DESKTOP-Igor:~/ex$ ./a.out
77 6
12.8333

```

friend функции

Функции могут быть другом сразу для нескольких классов.

```

1  #include <iostream>
2  /// информация компилятор, что определение Humidity будет
3  class Humidity;
4
5  class Temperature
6  {
7  private:
8      int m_temp;
9
10 public:
11     Temperature(const int temp = 0)
12     {
13         m_temp = temp;
14     }
15
16     friend void outWeather(const Temperature &temperature,
17                           const Humidity &humidity);
18 };
19
20 class Humidity
21 {
22 private:
23     int m_humidity;
24
25 public:
26     Humidity(const int humidity = 0)
27     {
28         m_humidity = humidity;
29     }
30
31     friend void outWeather(const Temperature &temperature,
32                           const Humidity &humidity);
33 };

```

```

35 void outWeather(const Temperature &temperature, const Humidity &humidity)
36 {
37     std::cout << "The temperature is " << temperature.m_temp <<
38               " and the humidity is " << humidity.m_humidity << '\n';
39 }
40
41 int main()
42 {
43     Temperature temp(15);
44     Humidity hum(11);
45
46     outWeather(temp, hum);
47     return 0;
48 }

```

```

chernousov@DESKTOP-Igor:~/ex$ g++ frined2.cpp
chernousov@DESKTOP-Igor:~/ex$ ./a.out
The temperature is 15 and the humidity is 11

```

friend классы

Один класс может быть дружественным другому классу, в таком случае все закрытые члены второго класса станут открытыми для первого класса.

friend классы

```

1  class Display;
2
3  class Values
4  {
5  private:
6      int m_intValue;
7      double m_dValue;
8
9  public:
10     Values(int intValue, double dValue) :
11         m_intValue(intValue),
12         m_dValue(dValue)
13     {
14     }
15
16     // Делаем класс Display другом класса Values
17     friend class Display;
18 };

```

```

1  #include "display.h"
2
3  int main()
4  {
5      Values value(7, 8.4);
6      Display display(false);
7      display.displayItem(value);
8      return 0;
9  }

```

```

1  #include <iostream>
2  #include "values.h"
3
4  class Display
5  {
6  private:
7      bool m_displayIntFirst;
8
9  public:
10     Display(bool displayIntFirst) :
11         m_displayIntFirst(displayIntFirst)
12     {
13     }
14
15     void displayItem(const Values &value)
16     {
17         if (m_displayIntFirst)
18             std::cout << value.m_intValue << " " << value.m_dValue << '\n';
19         else // или сначала выводим double
20             std::cout << value.m_dValue << " " << value.m_intValue << '\n';
21     }
22
23 };

```

```

chernousov@DESKTOP-Igor:~/ex$ g++ main.cpp
chernousov@DESKTOP-Igor:~/ex$ ./a.out
8.4 7
chernousov@DESKTOP-Igor:~/ex$ g++ main.cpp
chernousov@DESKTOP-Igor:~/ex$ ./a.out
7 8.4

```

friend классы

Важно:

- несмотря на то, что Display является другом Values, Display не имеет доступ к указателю *this объектов Values;
- из того, что Display является другом Values не значит, что Values является другом Display, если надо сделать оба класса дружественными друг другу, то необходимо явно указать в качестве друга противоположенный класс;
- если А является другом В, а В является другом С, это не значит, что А является другом С.

friend методы

Чтобы не делать весь класс дружественным другому, можно сделать дружественными только определенные методы класса.

Делается это аналогично объявлению дружественной функции за исключением имени метода с префиксом

`Имя_Класса::`

friend методы

```

1  #pragma once
2
3  #include "display.h"
4
5  class Values // полное определение класса Values
6  {
7  private:
8      int m_intValue;
9      double m_dValue;
10
11 public:
12     Values(int intValue, double dValue) :
13         m_intValue(intValue),
14         m_dValue(dValue)
15     {
16     }
17 }
18
19 // Делаем метод Display::displayItem() другом класса Values
20 friend void Display::displayItem(Values& value);
21 };

```

```

1  #include "values.h"
2
3  int main()
4  {
5      Values value; chernousov@DESKTOP-Igor:~/ex/friend class method$ g++ main.cpp display.cpp
6      Display displ; chernousov@DESKTOP-Igor:~/ex/friend class method$ ./a.out
7      display displ; 8.4 7
8      return 0; chernousov@DESKTOP-Igor:~/ex/friend class method$ g++ main.cpp display.cpp
9  } chernousov@DESKTOP-Igor:~/ex/friend class method$ ./a.out
    7 8.4

```

```

1  #pragma once
2
3  #include <iostream>
4
5  class Values;
6
7  class Display
8  {
9  private:
10     bool m_displayIntFirst;
11
12 public:
13     Display(bool displayIntFirst) :
14         m_displayIntFirst(displayIntFirst)
15     {
16     }
17
18     // предварительное объявление Values, приведенное выше, требуется для этой строки
19     void displayItem(Values &value);
20 };

```

```

1  #include "display.h"
2  #include "values.h"
3
4  void Display::displayItem(Values &value)
5  {
6      if (m_displayIntFirst)
7          std::cout << value.m_intValue << " " << value.m_dValue << '\n';
8      else // или выводим сначала double
9          std::cout << value.m_dValue << " " << value.m_intValue << '\n';
10 }

```

Дружественная функция/класс

Дружественная функция/класс – это функция/класс, которая имеет доступ к закрытым членам другого класса, как если бы она сама была членом этого класса. Это позволяет функции/классу работать в тесном контакте с другим классом, не заставляя другой класс делать открытыми свои закрытые члены.

Задача

Точка в геометрии – это позиция в пространстве. Точку можно определить в пространстве через набор координат x , y , z (`Point(0.0, 1.2, -2.0)`).

Вектор в физике – это величина, которая имеет длину и направление (но не положение). Можно определить вектор в пространстве через значения x , y и z , представляющие направление вектора вдоль осей x , y и z (`Vector(1.0, 0.0, 0.0)`).

Вектор может применяться к точке для перемещения точки на новую позицию. Это делается путем добавления направления вектора к позиции точки.

Например, `Point(0.0, 1.2, -2.0) + Vector(0.0, 2.0, 0.0)` даст точку `(0.0, 3.2, -2.0)`.

Точки и векторы часто используются в компьютерной графике (точка для представления вершин фигуры, а векторы — для перемещения фигуры).

1. Создать классы `Vector3d` и `Point3d`. Предусмотреть конструктор по умолчанию и конструктор с параметрами. Реализовать функции печати координат (`print`).
2. Сделать класс `Point3d` дружественным к `Vector3d` и реализовать метод `moveByVector()` в классе `Point3d`.
3. Вместо того, чтобы класс `Point3d` был дружественным к `Vector3d`, сделайте метод `Point3d::moveByVector()` дружественным классу `Vector3d`.

При выполнении используйте файловую декомпозицию.